DAY 03 | HANDS-ON ACTIVITIES

OOP and Design Principles: Problem Statements

Seven problems from warm-up to stretch. Solve them in the Day 3 notebook or in .py files in VS Code.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Student Class

Level: Warm-up | **Concepts:** class, __init__, methods

Create a Student class with name and a list of marks. Add methods add_mark(m) and average(). Add __str__ so print(student) shows name and average.

Sample run / expected result

```
s = Student("Aiko")
s.add_mark(80); s.add_mark(90)
print(s)   # Aiko (avg 85.0)
```

Hints

- Start marks as an empty list inside __init__.

Problem 2: Shapes and Polymorphism

Level: Easy | **Concepts:** inheritance, overriding

Create a Shape base class with area() that returns 0. Create Circle and Rectangle subclasses that override area(). Loop over a list of shapes and print each area.

Sample run / expected result

```
Circle 78.54
Rectangle 24
```

Hints

- type(obj).__name__ gives the class name.

Problem 3: Safe Division

Level: Easy | **Concepts:** try, except, else, finally

Write safe_divide(a, b) that returns the result, prints a friendly message for ZeroDivisionError and TypeError, and always prints "done" at the end.

Sample run / expected result

```
safe_divide(10, 2)   -> 5.0, done
safe_divide(10, 0)   -> Cannot divide by zero, done
```

Hints

- Use separate except blocks for each error type.

Problem 4: Bank Account with Rules

Level: Medium | **Concepts:** encapsulation, custom exceptions

Build BankAccount with deposit and withdraw. Raise a custom InsufficientFunds exception when withdrawing too much, and ValueError for negative amounts. Keep a transaction history list.

Sample run / expected result

```
acc.withdraw(5000)
InsufficientFunds: Balance 1200, requested 5000
```

Hints

- class InsufficientFunds(Exception): pass

Problem 5: Marks Analysis with NumPy

Level: Medium | **Concepts:** NumPy arrays, masks

Given marks for 10 students, use NumPy to print the mean, highest, lowest, standard deviation, and the marks above the mean.

Sample run / expected result

```
Mean: 71.3
Above mean: [78 85 92 74 88]
```

Hints

- np.array, .mean(), .std(), marks[marks > marks.mean()]

Problem 6: Timer Decorator

Level: Challenging | **Concepts:** decorators, *args, **kwargs

Write a @timer decorator that prints how many seconds a function took. Apply it to a function that sums the numbers up to 5 million.

Sample run / expected result

```
slow_sum took 0.21 s
12499997500000
```

Hints

- import time; start = time.perf_counter()

Problem 7: Library System

Level: Stretch | **Concepts:** classes, composition, exceptions, generators

Create Book and Library classes. Library has add_book, borrow(title) and return_book(title). Raise BookNotAvailable if a book is out. Add a generator available_books() that yields titles one at a time.

Sample run / expected result

```
lib.borrow("Ikigai")
lib.borrow("Ikigai")  -> BookNotAvailable
list(lib.available_books()) -> ['Atomic Habits']
```

Hints

- Library has a list of Book objects (composition).
- Each Book can have an is_available flag.

Ganbatte (頑張って): do your best